

# 3D Symbolic Region Puzzle

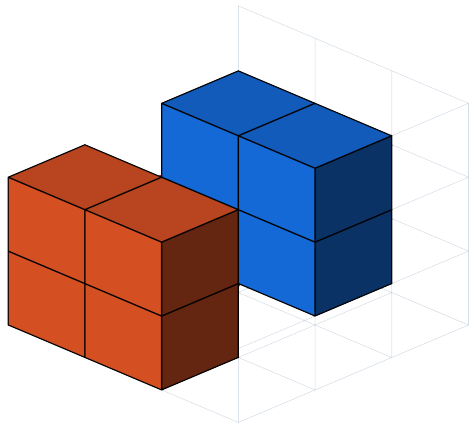
## A Non-Planar Symbolic Polycube Challenge

Odan Lafrance   Abdelouahad Alla

University of Mons — KRR 2026

May 2026

# A 3D Instance



## Puzzle characteristics:

- **Goal:** Partition a polycube of  $c$  unit cubes into exactly  $m$  non-planar regions without boundary violation.
- **Given:**
  - A 3D board shape
  - A finite set of drawn symbols  $\Sigma$
  - Physical walls  $W$
- **Method: Answer-set programming** (Clingo):  
Guess partition regions, compute spatial bounds, check constraints iteratively.

*View A: Non-Planar Instance*

# Puzzle Overview — Motivation

## Why ASP?

- Checking a candidate topological placement is fast.
  - Finding a correct structural placement is hard (combinations blow up).
- ⇒ ASP is a natural fit.

## Puzzle input

- Board geometry: Total cells  $c = 12$ , Boundaries  $N = 4$ .
- Sub-constraints: Number of regions  $m$ , classes of symbols  $k$ , walls  $|W|$ .

## Goal

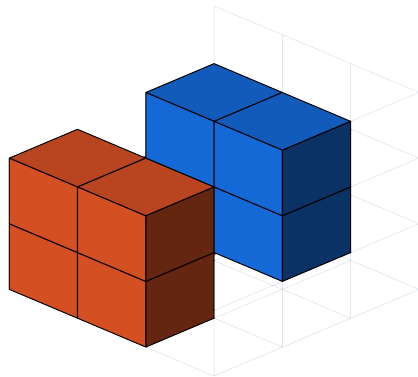
- Place non-planar partitions in the  $c$ -cube grid without geometric mismatch.
- Make the partitions strictly contain exactly one of each symbol  $\Sigma$ .

Context: Inspired by 3D polycube constraints and Golomb's original logic designs (1994).

# Puzzle Overview — The Constraints

## Rules properties:

- **Non-planar** — Not a flat “plate” of cubes, must expand across x, y, z individually per region!
- **Connectivity** — Pure spatial graph adjacency required.
- **Symbol Completeness** — A region acts as a strict container.
- **Wall Separation** — If a wall falls between two blocks, they belong to disjoint regions.



**Chirality / Volume matters here:**

This instance needs specific topological growth to avoid trapping uncolored cells.

# Puzzle Overview — Game Rules & Experience

## Input:

- Grid limits  $N$ , target size  $c$ .
- Visual markers (e.g., ★, ●) attached to unit shapes.

**Objective:** Place boundaries defining exactly  $m$  non-planar polycubes encapsulating everything. **Physical verification:**

- **Transparent/Painted Blocks** on tables.
- Check visually from outside (symbols always outward-facing).
- No tools, no breaking pieces. Done in seconds.

**Why difficult:** One single bad region placement starves surrounding adjacent cells of space, collapsing the entire grid recursively.

## Program split:

- **Map Generator:** Carves valid boards, places  $c$  cubes iteratively ensuring initial topological soundness and global bounding dimensions  $N$ .
- **Core solver** (guess / connect / prune).

## ASP Logical Pipeline:

- 1 **Guess:** One region ID per independent cube coordinate.
- 2 **Compute:** Breadth-first graph expansion checking adjacent ID similarities.
- 3 **Check:** Count spatial bounding box limits *per* aggregate ID (if  $\Delta z = 0$ , then prune).  
Verify exactly 1 symbol class  $k$  exists inside the aggregate ID. Identify overlapping walls.

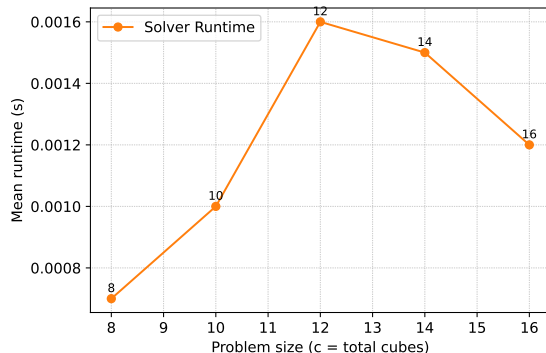
# Experiments & Complexity Checks

**Setup:** clingo 5.4+ · Apple Silicon

| $c$ | $m \times k$ | Walls | Mean     |
|-----|--------------|-------|----------|
| 8   | 4            | 2     | 0.0007 s |
| 10  | 4            | 2     | 0.0010 s |
| 12  | 6            | 3     | 0.0016 s |
| 14  | 6            | 3     | 0.0015 s |
| 16  | 8            | 4     | 0.0012 s |

## Key points:

- Heavily constrained rules lead to massive pruning.
- SAT extremely fast (fraction of ms) because the search tree is tight.



*Scaling of Core Solver Search Time*

## Complexity Analysis

- $\in$  NP (polynomial visual check)
  - Connectivity check is  $O(c)$ .
  - Non-planarity aggregates in  $O(c)$ .
  - Wall bounding is  $O(|W|)$ .
- Scales exponentially for random generation (generator constraints are incredibly sparse).

## Variants

- **Size hints:** Adding Fillomino-like region size parameters.

## Take-away

- 1 **Physical:** 3D printable, visually appealing.
- 2 **Verify:** Readily visible logic faces.
- 3 **Clingo:** Outstanding at recursively mapping non-trivial geometry intersections immediately.

## Next

- Extend map scale  $> 32$  cubes.
- Test optimization functions for solver symmetry breaking.